MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 24 — Visualisation de Données

Matplotlib, Seaborn, Plotly — transformer les données en insights

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

22 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

Chapitre 24 — Visualisation de Données **2**
0.1 L'écosystème de visualisation Python 2
0.2 Matplotlib — Les bases 2
0.3 Seaborn — Visualisation statistique 4
0.4 Plotly — Graphiques interactifs 6
0.5 Bonnes pratiques de visualisation 7
0.6 Ce qui m'a le plus surpris dans ce chapitre 9
0.7 Résumé du Chapitre 24 10

Chapitre 24 — Visualisation de Données

À retenir

La visualisation de données est l'art de transformer des chiffres en **compréhension**. Un graphique bien construit révèle des tendances, des anomalies et des relations invisibles dans un tableau brut. Python offre trois bibliothèques complémentaires : **Matplotlib** pour le contrôle total, **Seaborn** pour l'élégance statistique, et **Plotly** pour l'interactivité. Ce chapitre couvre les graphiques fondamentaux et les bonnes pratiques de la visualisation professionnelle.

L'écosystème de visualisation Python

Matplotlib
La fondation

Contrôle total
Tout type de graphique
Style très configurable
API bas niveau
Verbeux mais puissant

Seaborn
La statistique

Graphiques statistiques
Intégration Pandas native
Beaux par défaut
API haut niveau
Construit sur Matplotlib

Plotly
L'interactivité

Graphiques interactifs
HTML exportable
Zoom, hover, filtres
Dashboards avec Dash
Idéal pour le web

Matplotlib — Les bases

matplotlib — graphique en ligne

```python
import matplotlib.pyplot as plt
import numpy as np

# Graphique simple
x = np.linspace(0, 2*np.pi, 100)
y = np.sin(x)

plt.figure(figsize=(10, 5))
plt.plot(x, y, color="steelblue", linewidth=2,
         linestyle="-", label="sin(x)")
plt.plot(x, np.cos(x), color="tomato", linewidth=2,
         linestyle="--", label="cos(x)")

# Habillage
plt.title("Fonctions trigonometriques", fontsize=16,
          fontweight="bold", pad=15)
```

```python
plt.xlabel("x (radians)", fontsize=12)
plt.ylabel("Amplitude", fontsize=12)
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.axhline(0, color="black", linewidth=0.8)

plt.tight_layout()
plt.savefig("trigo.png", dpi=150, bbox_inches="tight")
plt.show()
```

matplotlib — subplots et types de graphiques

```python
import matplotlib.pyplot as plt
import numpy as np

fig, axes = plt.subplots(2, 3, figsize=(15, 8))
fig.suptitle("Galerie de graphiques Matplotlib",
             fontsize=16, fontweight="bold")

# 1. Courbe
ax = axes[0, 0]
x = np.linspace(0, 10, 200)
ax.plot(x, np.exp(-x/3)*np.sin(x*2),
        color="steelblue", linewidth=2)
ax.set_title("Courbe amortie")
ax.grid(True, alpha=0.3)

# 2. Nuage de points
ax = axes[0, 1]
np.random.seed(42)
x_s = np.random.randn(100)
y_s = 2*x_s + np.random.randn(100)*0.5
ax.scatter(x_s, y_s, c=y_s, cmap="viridis",
           alpha=0.7, s=50)
ax.set_title("Nuage de points")

# 3. Histogramme
ax = axes[0, 2]
data = np.random.normal(loc=15, scale=3, size=500)
ax.hist(data, bins=25, color="steelblue",
        edgecolor="white", alpha=0.8)
ax.axvline(data.mean(), color="red", linewidth=2,
           linestyle="--", label=f"Moy={data.mean():.1f}")
ax.legend()
ax.set_title("Histogramme")

# 4. Barres
ax = axes[1, 0]
categories = ["Python", "R", "Julia", "Scala", "Java"]
```

```
valeurs     = [72, 18, 5, 3, 2]
colors      = ["#2196F3","#4CAF50","#FF9800","#9C27B0","#F44336"
    ]
bars = ax.bar(categories, valeurs, color=colors, edgecolor="
    white")
for bar, val in zip(bars, valeurs):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height()
        +0.5,
            f"{val}%", ha="center", va="bottom", fontweight="
                bold")
ax.set_title("Langages Data Science")
ax.set_ylabel("Popularit  (%)")

# 5. Camembert
ax = axes[1, 1]
sizes  = [35, 25, 20, 15, 5]
labels = ["Pandas", "NumPy", "Sklearn", "Matplotlib", "Autres"]
explode = (0.05, 0, 0, 0, 0)
ax.pie(sizes, labels=labels, explode=explode,
       autopct="%1.1f%%", startangle=90,
       colors=["#2196F3","#FF9800","#4CAF50","#9C27B0","#607D8B
           "])
ax.set_title("Usage biblioth ques")

# 6. Boite    moustaches
ax = axes[1, 2]
data_box = [np.random.normal(m, 2, 50)
            for m in [10, 12, 15, 18]]
bp = ax.boxplot(data_box, labels=["G1","G2","G3","G4"],
                patch_artist=True)
colors_box = ["#2196F3","#4CAF50","#FF9800","#9C27B0"]
for patch, c in zip(bp["boxes"], colors_box):
    patch.set_facecolor(c)
    patch.set_alpha(0.7)
ax.set_title("Boites    moustaches")
ax.set_ylabel("Notes")

plt.tight_layout()
plt.savefig("galerie.png", dpi=150, bbox_inches="tight")
plt.show()
```

Seaborn — Visualisation statistique

seaborn — graphiques statistiques élégants

```
import seaborn as sns
import pandas as pd
import numpy as np
```

```
import matplotlib.pyplot as plt

# Donnees d'exemple
np.random.seed(42)
df = pd.DataFrame({
    "note":     np.concatenate([
                    np.random.normal(14, 2, 80),
                    np.random.normal(10, 3, 60)]),
    "groupe":   ["Avance"]*80 + ["D butant"]*60,
    "salaire":  np.random.normal(45000, 8000, 140),
    "exp":      np.random.randint(0, 15, 140),
    "ville":    np.random.choice(
                    ["Paris","Lyon","Marseille"], 140)
})
df["note"] = df["note"].clip(0, 20)

# Style global
sns.set_theme(style="whitegrid", palette="husl")

fig, axes = plt.subplots(2, 3, figsize=(16, 9))
fig.suptitle("Visualisations Seaborn", fontsize=16,
             fontweight="bold")

# 1. Distribution avec KDE
sns.histplot(data=df, x="note", hue="groupe",
             kde=True, ax=axes[0,0], alpha=0.6)
axes[0,0].set_title("Distribution des notes")

# 2. Violin plot
sns.violinplot(data=df, x="groupe", y="note",
               ax=axes[0,1], inner="box", palette="husl")
axes[0,1].set_title("Violin     notes par groupe")

# 3. Scatter avec r gression
sns.regplot(data=df, x="exp", y="salaire",
            ax=axes[0,2], scatter_kws={"alpha":0.5},
            line_kws={"color":"red"})
axes[0,2].set_title("R gression exp vs salaire")

# 4. Boxplot groupe
sns.boxplot(data=df, x="ville", y="salaire",
            hue="groupe", ax=axes[1,0], palette="Set2")
axes[1,0].set_title("Salaires par ville et groupe")

# 5. Heatmap de correlation
corr = df[["note","salaire","exp"]].corr()
sns.heatmap(corr, annot=True, fmt=".2f",
            cmap="coolwarm", ax=axes[1,1],
            square=True, linewidths=0.5)
axes[1,1].set_title("Matrice de corr lation")
```

```python

# 6. Countplot
sns.countplot(data=df, x="ville", hue="groupe",
              ax=axes[1,2], palette="husl")
axes[1,2].set_title("Effectifs par ville")

plt.tight_layout()
plt.savefig("seaborn_galerie.png", dpi=150,
            bbox_inches="tight")
plt.show()
```

Plotly — Graphiques interactifs

plotly — visualisations interactives

```python
import plotly.express as px
import plotly.graph_objects as go
import pandas as pd
import numpy as np

# Donnees
np.random.seed(42)
df = pd.DataFrame({
    "pays":    np.random.choice(
                  ["Togo","Cote d'Ivoire","Senegal",
                   "Ghana","Nigeria"], 100),
    "PIB":     np.random.uniform(500, 5000, 100),
    "IDH":     np.random.uniform(0.4, 0.8, 100),
    "pop":     np.random.uniform(1, 200, 100),
    "region":  np.random.choice(
                  ["Afrique de l'Ouest"], 100),
    "annee":   np.random.choice(range(2015, 2025), 100)
})

# Scatter interactif     hover, zoom, filtres
fig1 = px.scatter(
    df, x="PIB", y="IDH",
    size="pop", color="pays",
    hover_name="pays",
    title="PIB vs IDH     Afrique de l'Ouest",
    labels={"PIB": "PIB par habitant ($)",
            "IDH": "Indice de d veloppement humain"},
    size_max=40
)
fig1.update_layout(template="plotly_white")
fig1.write_html("scatter_interactif.html")
fig1.show()

```

```python
# Graphique en barres anime
fig2 = px.bar(
    df.groupby(["annee","pays"])["PIB"].mean().reset_index(),
    x="pays", y="PIB", color="pays",
    animation_frame="annee",
    title="Evolution PIB par pays",
    labels={"PIB": "PIB moyen ($)"}
)
fig2.write_html("barres_animees.html")

# Graphique combine avec go
fig3 = go.Figure()

x = np.linspace(0, 4*np.pi, 200)
fig3.add_trace(go.Scatter(
    x=x, y=np.sin(x),
    name="sin(x)", line=dict(color="#2196F3", width=2)
))
fig3.add_trace(go.Scatter(
    x=x, y=np.cos(x),
    name="cos(x)", line=dict(color="#F44336", width=2,
                             dash="dash")
))
fig3.update_layout(
    title="Fonctions trigonometriques     Interactif",
    xaxis_title="x", yaxis_title="f(x)",
    template="plotly_white",
    hovermode="x unified"
)
fig3.write_html("trigo_interactif.html")
fig3.show()
```

Bonnes pratiques de visualisation

Graphique professionnel — exemple complet

```python
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import numpy as np

# Palette coherente
BLEU   = "#1A3C6E"
ORANGE = "#E67E22"
VERT   = "#27AE60"
ROUGE  = "#E74C3C"
GRIS   = "#95A5A6"

fig = plt.figure(figsize=(14, 8),
```

```
                    facecolor="#FAFBFC")
gs  = gridspec.GridSpec(2, 2, figure=fig,
                        hspace=0.4, wspace=0.35)

# Titre global
fig.suptitle("Tableau de bord     Performance acad mique",
             fontsize=17, fontweight="bold",
             color=BLEU, y=0.98)

# --- Graphique 1 : Evolution temporelle
ax1 = fig.add_subplot(gs[0, :])   # pleine largeur
mois = ["Jan","Fev","Mar","Avr","Mai","Jun",
        "Jul","Aou","Sep","Oct","Nov","Dec"]
moy_2023 = [12.1,12.8,13.2,12.9,13.8,14.1,
            13.9,14.5,14.8,15.1,15.3,15.6]
moy_2024 = [13.5,13.9,14.2,14.8,15.1,15.6,
            15.4,16.0,16.2,16.5,16.8,17.0]

ax1.plot(mois, moy_2023, "o-", color=GRIS,
         linewidth=2, markersize=6, label="2023",
         alpha=0.7)
ax1.plot(mois, moy_2024, "o-", color=BLEU,
         linewidth=2.5, markersize=7, label="2024")
ax1.fill_between(mois, moy_2023, moy_2024,
                 alpha=0.1, color=BLEU)
ax1.axhline(10, color=ROUGE, linestyle="--",
            linewidth=1.5, alpha=0.7, label="Seuil admis")
ax1.set_title("  volution  mensuelle de la moyenne",
              fontsize=13, fontweight="bold", pad=10)
ax1.set_ylabel("Moyenne /20", fontsize=11)
ax1.legend(fontsize=10)
ax1.grid(True, alpha=0.3, linestyle="--")
ax1.set_facecolor("#F8F9FA")
for spine in ax1.spines.values():
    spine.set_color("#DEE2E6")

# --- Graphique 2 : Repartition mentions
ax2 = fig.add_subplot(gs[1, 0])
mentions = ["Tres Bien", "Bien", "Assez Bien",
            "Passable", "Echec"]
effectifs = [42, 68, 85, 45, 20]
couleurs  = [VERT, "#2ECC71", ORANGE, "#F39C12", ROUGE]
wedges, texts, autotexts = ax2.pie(
    effectifs, labels=mentions, colors=couleurs,
    autopct="%1.0f%%", startangle=90,
    pctdistance=0.75,
    wedgeprops={"edgecolor":"white","linewidth":2}
)
for at in autotexts:
    at.set_fontsize(9)
```

```
    at.set_fontweight("bold")
ax2.set_title("Repartition des mentions",
              fontsize=12, fontweight="bold", pad=10)

# --- Graphique 3 : Comparaison departements
ax3 = fig.add_subplot(gs[1, 1])
depts  = ["Info\nStat", " con \nAppl", "Math\nAppl",
          "Finance", "Socio"]
moyennes = [15.2, 13.8, 14.5, 12.9, 13.1]
bar_colors = [BLEU if m >= 14 else ORANGE
              for m in moyennes]
bars = ax3.barh(depts, moyennes, color=bar_colors,
                edgecolor="white", height=0.6)
for bar, val in zip(bars, moyennes):
    ax3.text(bar.get_width() + 0.1, bar.get_y() +
             bar.get_height()/2,
             f"{val:.1f}", va="center",
             fontsize=10, fontweight="bold")
ax3.axvline(14, color=ROUGE, linestyle="--",
            linewidth=1.5, alpha=0.7)
ax3.set_xlim(0, 18)
ax3.set_title("Moyennes par departement",
              fontsize=12, fontweight="bold", pad=10)
ax3.set_xlabel("Moyenne /20", fontsize=10)
ax3.set_facecolor("#F8F9FA")
ax3.grid(True, axis="x", alpha=0.3, linestyle="--")

plt.savefig("dashboard_academique.png", dpi=200,
            bbox_inches="tight", facecolor=fig.get_facecolor())
plt.show()
```

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. La dualité `plt` vs `axes` de Matplotlib. Matplotlib a deux interfaces : l'interface `plt.plot()` simple mais limitée, et l'interface orientée objet `fig, ax = plt.subplots()`. J'ai longtemps mélangé les deux. La règle d'or : pour tout graphique sérieux, toujours utiliser l'interface objet. Elle donne un contrôle total sur chaque élément, évite les comportements inattendus avec les subplots, et s'intègre proprement dans des fonctions réutilisables.

2. Seaborn et Pandas s'intègrent nativement. Passer directement un DataFrame à `sns.boxplot(data=df, x="ville", y="note")` est d'une élégance rare. Seaborn comprend la structure du DataFrame, gère les couleurs, les légendes et les labels automatiquement. Comparé à Matplotlib où il faut extraire chaque colonne manuellement, c'est une différence de niveaux d'abstraction qui change radicalement

la productivité en data exploration.

3. Plotly génère du HTML autonome. `fig.write_html("graphique.html")` crée un fichier HTML autonome — avec JavaScript embarqué, zoom interactif, hover tooltips — qu'on peut partager par email, publier sur LinkedIn, intégrer dans une page web, sans aucune dépendance Python côté destinataire. Pour un Data Scientist qui communique des résultats, c'est une capacité transformatrice.

Résumé du Chapitre 24

Ce que j'ai appris dans ce chapitre :

- **Matplotlib** : contrôle total, interface objet `fig, ax = plt.subplots()`.
- Courbes, scatter, histogrammes, barres, camembert, boxplots.
- `tight_layout()`, `savefig()`, habillage professionnel.
- **Seaborn** : `histplot`, `violinplot`, `regplot`, `heatmap`.
- Intégration native Pandas — `data=df, x="col", hue="cat"`.
- **Plotly Express** : graphiques interactifs en une ligne.
- `write_html()` — exporter un graphique interactif autonome.
- Bonnes pratiques : palette cohérente, titres clairs, grille légère.

"Un graphique bien fait dit en un regard ce qu'un tableau de mille lignes ne dit jamais. La visualisation, c'est la traduction des données en vérité."